

get_post_filter

REFERENCE

API consumer

read back the reader's current post-filter rules

Overview

`get_post_filter` reads back the reader's current post-filters — the rules that include or exclude scanned RFID tag data before it is reported to a data endpoint. Each rule matches tag data by pattern (prefix, suffix, or regex) and either reports (`INCLUDE`) or drops (`EXCLUDE`) it, scoped to a data endpoint. It is read-only. It is a **control-plane (CTRL) command**; configure filters with `set_post_filter`.

This is a **CTRL-endpoint command**: it is published on the **control** endpoint (`<tenantId>/CTRL/clients/cmdnd/<serial>`) and answered only there (not on the [MGMT/MDM](#) channel). Returning a response requires an **active CTRL endpoint** (`config_endpoint` , `epType: CTRL` , `activate: true`).

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	READ-ONLY
Required Request Fields	<code>command</code> , <code>requestId</code> (no other payload)
Applies To	Endpoint: Control (CTRL). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	<code>set_post_filter</code> , <code>get_operating_mode</code> , <code>config_endpoint</code>

When to Use

Use `get_post_filter` to inspect the reader's post-filters:

- Audit filtering** — see which patterns are included/excluded per data endpoint before relying on the tag-data stream.

- **After** `set_post_filter` — verify a filter was applied (the setter's response carries no configuration echo).
- **Confirm none are set** — an absent `postFilterPayload` means no filters are configured (see [Response Semantics](#)).

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/CTRL/clients/cmnd/<serial></code>	On the control endpoint. Requires an active CTRL endpoint .
Response	Reader → App	<code><tenantId>/CTRL/clients/resp/<serial></code>	Correlated reply; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Control commands are answered **only on the CTRL endpoint** — publishing `get_post_filter` to the MGMT/MDM command topic produces no reply.

Post-filters act on **tag data** (post-singulation), distinct from the Gen2 `select` pre-filters in `set_operating_mode` that act on the air interface. If control commands time out, confirm a CTRL endpoint is active and activate one with `config_endpoint` if needed.

See [Topics and Channels](#).

Key Fields

- `postFilterPayload.postFilter[]` (response) — the configured rules; absent when none are set.
- `dataEpType` (response) — the data endpoint (`DATA_EP1` / `DATA_EP2`) each rule scopes to. `DATA_EP2` targets the `DATA2` endpoint, which is currently not supported.
- `matchPattern` + `matchPatternMethod` (response) — what to match and how (`PREFIX` / `SUFFIX` / `REGEX`).
- `reportOperation` (response) — `INCLUDE` (report matches) or `EXCLUDE` (drop matches).
- `response.code` (response) — read this to determine success vs failure. The contract requires it; note the reported success-path omission (see [Success without a code](#)). See [Response Codes](#).

Request Payload

The request is a minimal envelope — just `command` and `requestId`. It returns all configured post-filters and takes no parameters.

Request payload

```
1 {
2   "command" : "get_post_filter" ,
3   "requestId" : "abc123"
4 }
```

Request payload

* required

— not applicable

Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_post_filter	Fixed command name.
2	requestId *	string	—	abc123	A unique identifier for the request; echoed back in the response. There is no other payload.

Response Payload

On success the device returns the envelope (`command` , `requestId` , `apiVersion` , `response`) plus `postFilterPayload` when filters are configured. The **Example** tab shows one `DATA_EP1` regex filter.

Response payload

Example: with a filter set

```
1 {
2   "command" : "get_post_filter" ,
3   "requestId" : "abc123" ,
4   "apiVersion" : "V1.1" ,
5   "postFilterPayload" : {
6     "postFilter" : [
7       {
8         "dataEpType" : "DATA_EP1" ,
9         "matchPattern" : "FFFF" ,
10        "matchPatternMethod" : "REGEX" ,
11        "reportOperation" : "INCLUDE"
12      }
13    ]
14  } ,
15  "response" : {
16    "code" : 0 ,
17    "description" : "Success"
18  }
19 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_post_filter	Echoes the executed command.
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
3	apiVersion [*]	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	postFilterPayload	object	—	—	The current post-filter configuration. Present only when at least one filter is configured — omitted entirely (not even an empty array) when none are set (see Response Semantics).
5	postFilter	array	—	—	The configured post-filter rules (one per entry).
6	dataEndpoint	enum	DATA_EP1 DATA_EP2	DATA_EP1	The data endpoint this filter applies to. DATA_EP2 targets the DATA2 endpoint, which is currently not supported.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
7	match pattern	string	—	FFFF	The pattern matched against scanned tag data.
8	match pattern Method	enum	PREFIX SUFFIX REGEX	REGEX	How the pattern is applied — prefix, suffix, or regular expression.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
9	report operation	enum	INCLUDE EXCLUDE	INCLUDE	Reports (INCLUDE) or drops (EXCLUDE) matching tag data.
10	response *	object	—	—	Status envelope for the read.
11	code *	integer	0-101	0	0 = Success; non-zero = error. Required by the contract. Readers have been reported to omit it on success — see Response Codes below.
12	description *	string	—	Success	Human-readable status.

When no filters are configured, the device omits `postFilterPayload` entirely:

Response — no filters configured

```
1  {
2    "command" : "get_post_filter" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "code" : 0 ,
7      "description" : "Success"
8    }
9  }
```

Response Semantics

- `postFilterPayload` is absent when no filters are set. With no post-filters configured, the device omits `postFilterPayload` entirely — not even an empty `postFilter: []`. Treat its absence as "no filters." When filters exist, `postFilter[]` lists them.

Response Codes

`get_post_filter` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the post-filter configuration was retrieved (an empty configuration still succeeds).
3	Not able to retrieve information (<code>IOT_ERROR_INFO_NOT_AVAILABLE</code>) — the device could not gather the post-filters at this time. Retry after a short delay; if persistent, reboot.
101	Default error — generic catch-all.

The contract requires `response.code`. Readers have been reported to omit it on a successful reply, returning just `response: { "description": "Success" }` — treat that as success too. See [Success without a code](#).

Examples

Publish `get_post_filter` over the CTRL endpoint (with an active CTRL endpoint); the device replies with the current filters. The [Response Payload](#) shows a configured filter; the no-filter reply is shown above it. There are no request variants — the command takes no parameters.

Notes and Caveats

- **Post-filters vs pre-filters.** `get_post_filter` reports **post-filters** — pattern rules applied to scanned **tag data** before it reaches a data endpoint (`INCLUDE` / `EXCLUDE`). These are separate from the Gen2 `select` **pre-filters** in `set_operating_mode`, which filter on the air interface during inventory. Configure post-filters with `set_post_filter`.

`get_post_filter` is a read-only, idempotent getter — safe to poll. Use it to verify a change applied by `set_post_filter`; it carries no credentials in request or response. For the concept-level view, see [Tag Filtering](#).

Related:  `set_post_filter` ·  `get_operating_mode` ·  `config_endpoint` ·  [Tag Filtering](#) ·  [Response Codes](#)